Nama : Muhammad nadzar al ghifary
Kelas :TI 1H
No. Presensi : 23

Pertanyaan percobaan 1

1. Program akan mengalami error (gagal dikompilasi). Hal ini terjadi karena array bil dideklarasikan dengan tipe data int (integer), yang hanya dapat menyimpan bilangan bulat. Angka seperti 5.0 dan 7.5 adalah bilangan pecahan (tipe data double/float), sehingga Java akan menolak untuk memasukkannya ke dalam array int secara langsung (terjadi type mismatch).
2. int[] bil = {5, 13, -7, 17};
3. • Keluaran program: Program akan mencetak indeks array, nilai asli dari elemen tersebut, serta hasil perkalian nilai elemen tersebut dengan angka 2. Contohnya untuk indeks ke-0 akan tercetak: bil[0] = 5, double = 10.
   • Maksud statement: Statement tersebut berfungsi untuk menampilkan informasi teks gabungan (string concatenation) yang berisi nomor indeks (i), nilai elemen saat itu (bil[i]), serta hasil operasi aritmatika berupa nilai elemen dikalikan dua (bil[i] * 2) secara langsung di dalam perintah cetak
4. Program akan berjalan normal pada awal perulangan, namun pada iterasi terakhir (i = 4) program akan mengalami error saat dijalankan (runtime error) yaitu muncul pesan ArrayIndexOutOfBoundsException. Hal ini terjadi karena array bil hanya memiliki kapasitas 4 elemen dengan indeks dari 0 hingga 3. Ketika i = 4, program mencoba mengakses bil[4] yang posisinya sudah berada di luar batas alokasi memori array.
5. for loop standar: Menggunakan kendali indeks (i). Cocok digunakan ketika kita membutuhkan akses ke posisi indeks tertentu (misalnya untuk manipulasi data berdasarkan indeks atau menampilkan nomor indeksnya seperti bil[i]).
   foreach loop: Digunakan untuk mengakses seluruh elemen array secara berurutan tanpa perlu memikirkan nomor indeks atau batas panjang array. Cocok digunakan saat kita hanya ingin membaca (read) seluruh isi nilai array secara praktis.
   Arrays.toString(): Merupakan fungsi bawaan dari Java (java.util.Arrays) yang langsung mencetak seluruh isi array dalam format satu baris berbentuk tanda kurung siku dan dipisahkan koma (contoh: [5, 13, 7, 17]). Sangat cocok digunakan untuk keperluan debugging atau menampilkan isi array secara cepat dan ringkas.

Pertanyaan percobaan 2

1. Kondisi tersebut digunakan sebagai batas atau syarat agar perulangan (for loop) terus berjalan. Variabel i akan terus bertulang (increment) selama nilainya masih lebih kecil dari panjang total elemen di dalam array nilaiAkhir.length. Jika i sudah mencapai atau melebihi jumlah elemen array, maka perulangan akan berhenti secara otomatis untuk menghindari error *ArrayIndexOutOfBoundsException*.
2. Untuk menyaringnya, kita cukup menambahkan struktur percabangan if di dalam perulangan for sebelum perintah cetak.

Contoh Kode:

Java
```
for (int i = 0; i < nilaiAkhir.length; i++) {
    if (nilaiAkhir[i] > 70) {
        System.out.println("Mahasiswa ke-" + (i + 1) + " lulus dengan nilai: " +
nilaiAkhir[i]);
    }
}
```

Alur Program:

Perulangan for mengecek satu per satu elemen array dari indeks 0 sampai akhir.

Di dalam perulangan, terdapat seleksi kondisi if (nilaiAkhir[i] > 70).

Jika nilai pada indeks tersebut lebih besar dari 70, program akan mengeksekusi perintah System.out.println untuk mencetak data mahasiswa yang lulus.

Sebaliknya, jika nilai <= 70, kondisi bernilai false, sehingga program mengabaikannya dan langsung lanjut ke iterasi berikutnya tanpa menampilkan apa pun ke terminal.

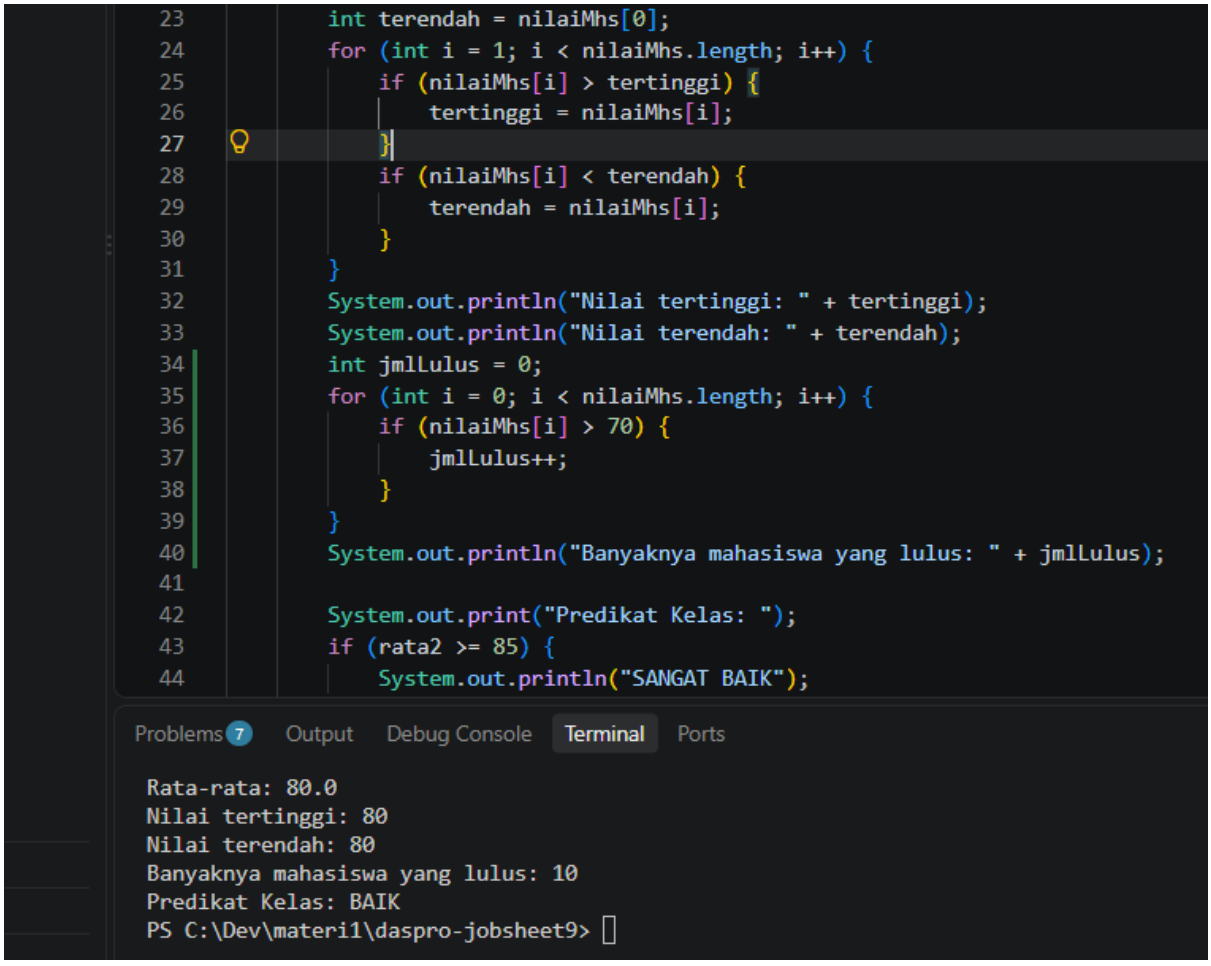
3. Bagian yang BISA diubah ke foreach: Perulangan untuk menampilkan isi array atau membaca data (misalnya saat mencetak daftar nilai atau melakukan pengecekan kelulusan). Contohnya: for (int nilai : nilaiAkhir) { ... }.
   Bagian yang TIDAK BISA diubah ke foreach: Perulangan untuk mengisi nilai ke dalam array (input dari pengguna menggunakan Scanner) atau ketika kita butuh mengakses nomor indeks (i) secara spesifik (seperti menampilkan "Mahasiswa ke-1", "Mahasiswa ke-2", dsb., atau melakukan modifikasi nilai elemen di dalam array).
   Alasannya: Foreach loop bersifat read-only (hanya bisa membaca) dan tidak menyediakan variabel indeks i secara otomatis. Karena foreach tidak melacak posisi indeks, kita tidak bisa menggunakannya untuk mengisi data ke indeks tertentu atau mengetahui posisi elemen ke-berapa yang sedang diakses.
4. Kita bisa menambahkan dua buah variabel penghitung (*counter*), misalnya jmlLulus dan jmlTidakLulus, yang nilainya akan bertambah (++) setiap kali kondisi kelulusan terpenuhi di dalam perulangan.
   Java
   ```
   int jmlLulus = 0;
   int jmlTidakLulus = 0;

   for (int i = 0; i < nilaiAkhir.length; i++) {
       if (nilaiAkhir[i] > 70) {
           jmlLulus++;
       } else {
           jmlTidakLulus++;
       }
   ```

```
}

System.out.println("Jumlah mahasiswa yang lulus: " + jmlLulus);
System.out.println("Jumlah mahasiswa yang tidak lulus: " + jmlTidakLulus);
```

Pertanyaan percobaan 3

1. 

2. Di github

3. Digunakan type casting (double) agar hasil pembagiannya menghasilkan bilangan desimal (pecahan) yang akurat.

   Jika langsung ditulis total / nilaiMhs.length (di mana kedua variabel bertipe data int atau integer), Java akan melakukan pembagian bulat (integer division), sehingga sisa atau angka di belakang koma akan dihilangkan secara otomatis. Dengan menambahkan (double), operasi pembagian tersebut dipaksa menjadi operasi desimal sehingga nilai rata-rata tampil dengan presisi yang benar.

Pertanyaan Percobaan4

1. Statement break; digunakan untuk **menghentikan perulangan (for loop) secara paksa** begitu nilai yang dicari (key) sudah ditemukan di dalam array. Hal ini membuat program berjalan lebih efisien karena tidak perlu melanjutkan pengecekan ke sisa indeks array di belakangnya jika data yang dicari sudah ketemu.
2. Di github
3. Modifikasi pesan dapat dilakukan pada blok pengecekan else seperti berikut:
   ```
   if (ditemukan) {
       System.out.println("Nilai " + key + " ditemukan pada indeks ke-" + indeksHasil);
   } else {
       System.out.println("Maaf, Nilai \"" + key + "\" tidak ditemukan di dalam array. Silakan coba masukkan nilai lain.");
   }
   ```
4. Dengan menggunakan data awal uji coba: [78, 65, 90, 55, 88, 72], algoritma Bubble Sort (ascending) bekerja dengan membandingkan dua elemen yang berdampingan secara berpasangan dan menukar posisinya jika urutannya salah (elemen kiri lebih besar dari elemen kanan).

   Tahap 1 (Iterasi Pertama):

   a. Bandingkan 78 dan 65 → Karena $78 > 65$, tukar posisi → Array menjadi: [65, 78, 90, 55, 88, 72]
   b. Bandingkan 78 dan 90 → $78 < 90$ (posisi sudah benar, tidak ditukar).
   c. Bandingkan 90 dan 55 → Tukar posisi → Array: [65, 78, 55, 90, 88, 72]
   d. Bandingkan 90 dan 88 → Tukar posisi → Array: [65, 78, 55, 88, 90, 72]
   e. Bandingkan 90 dan 72 → Tukar posisi → Array: [65, 78, 55, 88, 72, 90] (Nilai terbesar yaitu 90 kini "menggelembung" dan bergeser ke posisi paling akhir).

   Tahap Berikutnya (Iterasi ke-2 hingga selesai): Proses perbandingan berulang kembali untuk elemen-elemen di depannya dengan mengabaikan elemen terakhir yang sudah terurut. Langkah ini terus diulang sampai seluruh array tersusun rapi dari nilai terkecil hingga terbesar menjadi: [55, 65, 72, 78, 88, 90].

5. Di github